EJERCICIO 13 - Red del Servicio Nacional de Sangre

Grafo: ver ej13_grafo.png.
Código: RedSangre (con colecciones de Java en lugar de TDALista y THash),
Envio, MainEjercicio13 (responde la parte 1.3) y Ejercicio13Test.

En los órdenes, V es la cantidad de vértices (bancos + hospitales), E la
cantidad de trayectos, B la cantidad de bancos y H la de hospitales.

==
PARTE 1 - MODELADO Y SEUDOCÓDIGO
==

----
1. MODELADO

- Vértices: los 7 puntos de la red, los 3 bancos (BC, BN, BE) y los 4
  hospitales (HC, HP, HM, HE). Se identifican por su sigla (String).

- Aristas: cada trayecto que hace una ambulancia, de un punto a otro, con los
  minutos que tarda como peso.

- Tipo de grafo: DIRIGIDO y PONDERADO, con pesos no negativos.
  * Dirigido porque las ambulancias respetan el sentido de las calles: ir de
    A a B no siempre tarda lo mismo que volver, y a veces no se puede volver
    directo (hay BC -> HM pero no HM -> BC).
  * Ponderado porque lo que importa es el tiempo, no la cantidad de tramos.
  * No es fuertemente conexo: por ejemplo, de HE no sale ningún trayecto.
  * Pesos >= 0: se puede usar Dijkstra.

- Estructuras:
  * red: IDirectedIGraph<String, WeightedEdge> con lista de adyacencias
    (DirectedGraph). La red es dispersa (11 trayectos de 42 posibles), así
    que la lista ocupa O(V + E) en vez de O(V²), y Dijkstra recorre las
    aristas salientes de cada vértice, que es justo lo que da la lista.
  * redInversa: el mismo grafo con cada trayecto dado vuelta. Para abastecer
    hace falta lo que tarda CADA BANCO en llegar a UN hospital (caminos
    mínimos hacia un destino). En la red invertida eso es un solo Dijkstra
    desde el hospital. Ocupa O(V + E) más y se arma una vez, al cargar la red.
  * bancos y hospitales: listas en orden de registro, para recorrerlos y para
    desempatar (si dos bancos tardan lo mismo, va primero el registrado antes).
  * stock: hash banco -> unidades. Se consulta y se modifica en cada pedido,
    y el hash lo hace en O(1) promedio.
  * Caminos ya calculados: como hay muchos pedidos por día y los tiempos no
    dependen del stock, se guardan:
      - por hospital, el resultado del Dijkstra sobre la red invertida y la
        lista de bancos que llegan, ordenada por tiempo;
      - el menor tiempo desde algún banco hasta cada hospital.
    Se borran solo si cambia la red (se agrega un banco, un hospital o un
    trayecto). Así cada camino mínimo se calcula una sola vez.
  * Envio: (banco, unidades, minutos).

----
2. ALGORITMOS

2.1 abastecer(hospital, unidades, maxMinutos)

Lenguaje natural:
Se obtienen los bancos que llegan al hospital, ordenados de menor a mayor tiempo (la primera vez con un Dijkstra desde el hospital sobre la red invertida; después se reutiliza). Primero se suma el stock de los bancos que llegan en maxMinutos o menos: si no alcanza para el pedido, se devuelve la lista vacía sin tocar el stock. Si alcanza, se recorren esos bancos en orden y de cada uno se saca lo que tiene o lo que falta, lo que sea menor, hasta completar el pedido. Cada saque se anota como envío y se descuenta del stock.

Precondiciones:
- Los minutos de los trayectos son >= 0.
- Si hospital no es un hospital registrado o unidades <= 0, se devuelve la lista vacía.

Postcondiciones:
- La red no cambia.
- Si devuelve la lista vacía, el stock no cambió.
- Si no:
  * la suma de las unidades de los envíos es exactamente unidades;
  * cada envío sale de un banco que llega en maxMinutos o menos;
  * los envíos están en orden de tiempo creciente y ningún banco se repite;
  * un banco solo se usa si los más rápidos quedaron sin stock;
  * el stock de cada banco usado bajó en lo que envió y no es negativo.

Pseudocódigo:

```
abastecer(hospital, unidades, maxMinutos) devuelve Lista de Envio
Comienzo
    envios <- lista vacía
    SI hospital NO ESTA EN hospitales O unidades <= 0 ENTONCES
        devolver envios
    FIN SI

    orden <- bancosPorTiempo(hospital)
    tiempos <- tiemposHacia.buscar(hospital)

    // primero se ve si alcanza, sin tocar el stock
    disponibles <- 0
    PARA CADA banco EN orden HACER
        SI tiempos.costo(banco) > maxMinutos ENTONCES
            salir del PARA                  // los que siguen tardan más
        FIN SI
        disponibles <- disponibles + stock.buscar(banco)
    FIN PARA
    SI disponibles < unidades ENTONCES
        devolver envios
    FIN SI

    falta <- unidades
    i <- 0
    MIENTRAS i < orden.tamaño() Y falta > 0 HACER
        banco <- orden.obtener(i)
        cantidad <- mínimo(stock.buscar(banco), falta)
        SI cantidad > 0 ENTONCES
            envios.agregar(nuevo Envio(banco, cantidad, tiempos.costo(banco)))
            stock.insertar(banco, stock.buscar(banco) - cantidad)
            falta <- falta - cantidad
        FIN SI
        i <- i + 1
```

```
    FIN MIENTRAS
    devolver envios
Fin

(Como lo disponible en maxMinutos alcanza, falta llega a 0 antes de pasar a
un banco que tarde más de maxMinutos.)

bancosPorTiempo(hospital) devuelve Lista de bancos
Comienzo
    SI bancosPorTiempo.contiene(hospital) ENTONCES
        devolver bancosPorTiempo.buscar(hospital)
    FIN SI
    tiempos <- dijkstra(hospital, redInversa)   // tiempo de cada vértice HACIA el hospital
    orden <- lista vacía
    PARA CADA banco EN bancos HACER
        SI tiempos.costo(banco) <> infinito ENTONCES
            orden.agregar(banco)
        FIN SI
    FIN PARA
    ordenar orden por tiempos.costo(banco), estable (en empate, orden de registro)
    tiemposHacia.insertar(hospital, tiempos)
    bancosPorTiempo.insertar(hospital, orden)
    devolver orden
Fin
```

Orden del tiempo de ejecución:
- Primer pedido de un hospital: Dijkstra $O((V + E) \log V)$ más ordenar los bancos $O(B \log B)$. En esta implementación, cada vez que Dijkstra pide las adyacencias de un vértice lo busca con un Comparable ($O(V)$), así que el Dijkstra queda $O(V^2 + E \log V)$.
- Los pedidos siguientes de ese hospital: $O(H + B)$ (ver si es hospital en la lista y recorrer los bancos), sin ningún camino mínimo.
- Comparación: hacer un Dijkstra por banco en cada pedido costaría B veces el Dijkstra por pedido.

2.2 hospitalesSinCobertura(maxMinutos)

Lenguaje natural:
Un hospital está cubierto si algún banco llega en maxMinutos o menos, sin importar el stock. La primera vez se hace un Dijkstra desde cada banco sobre la red (no la invertida) y se guarda, para cada hospital, el menor tiempo desde algún banco. Después alcanza con recorrer los hospitales y quedarse con los que tienen ese menor tiempo mayor que maxMinutos.

Precondiciones:
- Los minutos de los trayectos son >= 0.

Postcondiciones:
- La red y el stock no cambian.
- Devuelve, en orden de registro, los hospitales h tales que para todo banco b el tiempo mínimo de b a h es mayor que maxMinutos (o no hay camino).

Pseudocódigo:

```
hospitalesSinCobertura(maxMinutos) devuelve Lista
Comienzo
    SI mejorTiempo = nulo ENTONCES
        calcularMejorTiempo()
    FIN SI
```

```
    sinCobertura <- lista vacía
    PARA CADA h EN hospitales HACER
        SI mejorTiempo.buscar(h) > maxMinutos ENTONCES
            sinCobertura.agregar(h)
        FIN SI
    FIN PARA
    devolver sinCobertura
Fin

calcularMejorTiempo()
Comienzo
    mejorTiempo <- nuevo hash
    PARA CADA h EN hospitales HACER
        mejorTiempo.insertar(h, infinito)
    FIN PARA
    PARA CADA banco EN bancos HACER
        desdeBanco <- dijkstra(banco, red)      // una sola vez por banco
        PARA CADA h EN hospitales HACER
            mejorTiempo.insertar(h, mínimo(mejorTiempo.buscar(h), desdeBanco.costo(h)))
        FIN PARA
    FIN PARA
Fin
```

Orden del tiempo de ejecución:
- Primera consulta: B Dijkstras, O(B · (V + E) log V), más O(B · H) para quedarse con los mínimos.
- Consultas siguientes: O(H).
- Mejora posible: un Dijkstra con varios orígenes (todos los bancos empiezan con costo 0 en la cola) da el menor tiempo desde algún banco en una sola pasada, O((V + E) log V).

---

3. RESPUESTAS

3.1 El Hospital de Clínicas pide 8 unidades con 30 minutos como máximo.

Dijkstra desde HC sobre la red invertida (tiempo de cada punto HACIA HC):

Paso	Se fija	Actualiza
0	HC = 0	HM 8, BC 25, BE 50, HP 20
1	HM = 8	BC 8 + 12 = 20 (mejora 25), BE 8 + 30 = 38 (mejora 50)
2	BC = 20	BN 20 + 10 = 30
3	HP = 20	BN 20 + 15 = 35: no mejora 30
4	BN = 30	HM: ya fijo
5	BE = 38	HC: ya fijo

HE no se alcanza: no sale ningún trayecto de HE.

Tiempos de los bancos a HC:

Banco	Minutos	Recorrido	Stock
BC	20	BC -> HM -> HC	6
BN	30	BN -> BC -> HM -> HC	8
BE	38	BE -> HM -> HC	16

Con 30 minutos como máximo sirven BC y BN (BN llega justo en 30). Entre los dos tienen 14 unidades, que alcanzan para 8.
- BC: tiene 6, faltan 8 -> se sacan 6, faltan 2.
- BN: tiene 8, faltan 2 -> se sacan 2, faltan 0.

El sistema envía: (BC, 6 unidades, 20 min) y (BN, 2 unidades, 30 min).
Stock después: BC 0, BN 6, BE 16.

3.2 Hospitales sin cobertura con 15 minutos como máximo.

Dijkstra desde cada banco sobre la red (tiempo DESDE el banco):

Desde	HC	HP	HM	HE
BC	20	45	12	39
BN	30	15	22	49
BE	38	63	30	14
Mínimo	20	15	12	14

- HC: el más cercano es BC con 20 > 15 -> SIN COBERTURA.
- HP: BN llega en 15 (BN -> HP)   -> cubierto.
- HM: BC llega en 12 (BC -> HM)   -> cubierto.
- HE: BE llega en 14 (BE -> HE)   -> cubierto.

Queda sin cobertura solo el Hospital de Clínicas (HC).

MainEjercicio13 da los mismos resultados.

==
PARTE 2 - CORRECCIÓN DE CÓDIGO
==

Los números de línea son los de la hoja. Bancos en orden de registro:
BC, BN, BE. Hospitales: HC, HP, HM, HE.

----
abastecer

ERROR 1 - Línea 11: Dijkstra dentro del for y sobre la red equivocada.
  Qué está mal:
  a) Está dentro del for de los bancos: se hace el mismo Dijkstra desde el
     hospital una vez por banco (3 veces en lugar de 1). Repite cálculos de
     caminos mínimos, que es lo que pide evitar el enunciado.
  b) dijkstra(hospital, red) da lo que tarda el HOSPITAL en llegar a cada
     banco, no lo que tarda cada banco en llegar al hospital.
  Dónde se nota: HC con 8 unidades y 30 minutos (con los demás errores
  corregidos). En la red, desde HC se llega
  a BE en 5 (HC -> BE), a BN en 53 y a BC en 63. El único candidato es BE,
  "a 5 minutos", y se mandan las 8 unidades desde BE. Pero BE tarda 38 en
  llegar a HC: la sangre llega tarde. Lo correcto es mandar 6 de BC y 2 de BN.
  Corrección: sacar la línea 11 del for y usar la red invertida. Antes de la
  línea 9:
      IDijkstraResult<String> tiempos = algoritmos.dijkstra(hospital, redInversa);
  y borrar la línea 11.

ERROR 2 - Línea 13: t < maxMinutos.
  Qué está mal: deja afuera al banco que llega justo en el tiempo máximo, y
  esa sangre sirve (no llega "después").
  Dónde se nota: HC con 8 unidades y 30 minutos (con los demás errores
  corregidos). BN tarda exactamente 30 y
  queda afuera. Con BC solo hay 6 unidades, el pedido no se cubre y no se
  envía nada.
  Corrección: if (t <= maxMinutos) {

ERROR 3 - Línea 24: int cantidad = disponibles;
  Qué está mal: saca todo el stock del banco aunque falte menos.
  Dónde se nota: HC con 8 unidades y 30 minutos (con los errores 1 y 2 ya
  corregidos). Se sacan 6 de BC y 8 de BN: se envían 14 unidades en lugar de
  8, BN queda en 0 y falta termina en -6.
  Corrección: int cantidad = Math.min(disponibles, falta);

ERROR 4 - Líneas 27 y 32-33: el stock se descuenta antes de saber si alcanza.
  Qué está mal: la línea 27 descuenta el stock de cada banco mientras arma los
  envíos. Si al final falta > 0, las líneas 32-33 devuelven la lista vacía
  ("no se envía nada"), pero el stock ya quedó descontado: esas unidades se
  pierden.
  Dónde se nota: HC con 20 unidades y 30 minutos. BC y BN tienen 14 en total:
  no alcanza, se devuelve la lista vacía, pero BC y BN quedan en 0.
  Corrección: antes de descontar, sumar el stock de los candidatos y, si no
  alcanza, devolver la lista vacía sin tocar nada. Por ejemplo, en el primer
  for (línea 14-15) acumular total = total + stock.buscar(b), y antes de la
  línea 20:
      if (total < unidades) {
         return envios;
      }
  Con eso las líneas 32-34 sobran.

Código corregido:

```
 public TDALista<Envio> abastecer(String hospital, int unidades, double maxMinutos) {
    TDALista<Envio> envios = new ListaSimplementeEnlazada<>();
    if (!red.existeVertice(hospital) || unidades <= 0) {
       return envios;
    }

    THash<String, Double> tiempoDe = new HashSondeoLineal<>(16);
    TDALista<String> candidatos = new ListaSimplementeEnlazada<>();
    IDijkstraResult<String> tiempos = algoritmos.dijkstra(hospital, redInversa);   // ERROR 1
    int total = 0;                                                                 // ERROR 4
    for (int i = 0; i < bancos.tamaño(); i++) {
       String b = bancos.obtener(i);
       double t = tiempos.getCost(b);
       if (t <= maxMinutos) {                                                      // ERROR 2
          candidatos.agregar(b);
          tiempoDe.insertar(b, t);
          total = total + stock.buscar(b);                                         // ERROR 4
       }
    }
    if (total < unidades) {                                                        // ERROR 4
       return envios;
    }
    candidatos = candidatos.ordenar(Comparator.comparingDouble(b -> tiempoDe.buscar(b)));

    int falta = unidades;
    for (int i = 0; i < candidatos.tamaño() && falta > 0; i++) {
       String b = candidatos.obtener(i);
       int disponibles = stock.buscar(b);
       int cantidad = Math.min(disponibles, falta);                                // ERROR 3
       if (cantidad > 0) {
          envios.agregar(new Envio(b, cantidad, tiempoDe.buscar(b)));
          stock.insertar(b, disponibles - cantidad);
          falta = falta - cantidad;
       }
```

```
    }
    return envios;
 }
```

Observación (no la contamos como error): la línea 3 acepta cualquier vértice de la red, también un banco. Si se pide abastecer("BC", ...), BC se toma a sí mismo como candidato con tiempo 0. Si la precondición no garantiza que sea un hospital, conviene preguntar hospitales.contiene(hospital).

---

hospitalesSinCobertura

ERROR 5 - Línea 44: Dijkstra dentro del for de hospitales y sobre la red equivocada.

Qué está mal:

a) El Dijkstra de cada banco se repite una vez por hospital: 4 · 3 = 12 Dijkstras cuando alcanza con 3, uno por banco. Repite cálculos.

b) dijkstra(banco, redInversa) da lo que tarda cada punto en llegar AL BANCO, no lo que tarda el banco en llegar al hospital.

Dónde se nota: con 15 minutos (y el error 6 corregido). En la red invertida, desde BE da HC = 5 (es lo que tarda HC en llegar a BE), así que HC aparece cubierto; y HE da infinito para todos los bancos (de HE no sale nada), así que HE aparece sin cobertura aunque BE llega en 14. Resultado: HP y HE en lugar de solo HC.

Corrección: calcular un Dijkstra por banco sobre red, una sola vez, antes del for de la línea 40, y usarlos dentro:

```
TDALista<IDijkstraResult<String>> desdeBancos = new ListaSimplementeEnlazada<>();
for (int j = 0; j < bancos.tamaño(); j++) {
   desdeBancos.agregar(algoritmos.dijkstra(bancos.obtener(j), red));
}
```

ERROR 6 - Línea 45: cubierto = desdeBanco.getCost(h) <= maxMinutos;

Qué está mal: cada vuelta pisa el valor anterior, así que al final cubierto solo dice si el ÚLTIMO banco (BE) llega a tiempo.

Dónde se nota: con 15 minutos (y el error 5 corregido). HP está cubierto por BN (15) y HM por BC (12), pero BE tarda 63 y 30 respectivamente: los dos aparecen sin cobertura. Resultado: HC, HP y HM en lugar de solo HC.

Corrección: cubierto = cubierto || desdeBanco.getCost(h) <= maxMinutos;

o, mejor, cortar el for apenas un banco llega (j < bancos.tamaño() && !cubierto).

Código corregido:

```
 public TDALista<String> hospitalesSinCobertura(double maxMinutos) {
    TDALista<String> sinCobertura = new ListaSimplementeEnlazada<>();
    TDALista<IDijkstraResult<String>> desdeBancos = new ListaSimplementeEnlazada<>();   // ERROR 5
    for (int j = 0; j < bancos.tamaño(); j++) {
       desdeBancos.agregar(algoritmos.dijkstra(bancos.obtener(j), red));
    }
    for (int i = 0; i < hospitales.tamaño(); i++) {
       String h = hospitales.obtener(i);
       boolean cubierto = false;
       for (int j = 0; j < desdeBancos.tamaño() && !cubierto; j++) {                    // ERROR 6
          cubierto = desdeBancos.obtener(j).getCost(h) <= maxMinutos;
       }
       if (!cubierto) {
          sinCobertura.agregar(h);
       }
    }
```

```
    return sinCobertura;
 }
```

RedSangre implementa esta misma lógica con colecciones de Java y, además, guarda los caminos mínimos entre llamadas (parte 1). Ejercicio13Test prueba los casos de arriba: el límite inclusivo (30), que se saca solo lo que falta, que un pedido que no se cubre no toca el stock, que el stock se descuenta entre pedidos y la cobertura con 10, 14, 15 y 20 minutos.